

Revised architecture

It appears from what I see when I click into request queue and open a record, (noting the progress bar at the top labelled processing pipeline) that the design was created in a way that combined processing of tasks and status of events not associated with actual work done in processing tasks, and the bar applies to the failed model of the prior data schema.

The parent child data schema for requests recently implemented will allow for processing and design to be more consistent, coherent, and user friendly.

Keep in mind that a parent record is conceptually not processed, with exception of estimate/payment and a few other items....Any processing of a request is at the child level where the description exists.

The workflow for processing child request items is to be driven by task completion. Exclude MRR for the moment and consider only single item request processing. The typical path would involve intake review, estimate information collection, records search, redaction/legal redaction/delivery (ignoring review tasks at the moment for simplicity). I have not thought through the branch or an item request that triggers legal review, and will do that asap. At the parent level, for simplicity, assume that the status of a parent is either active or closed, or hold awaiting payment. Closed at the parent level would typically be driven by a terminal status event at the child level....no response to request for clarification within statutory timeline would be one. No matching records would be another. However, in the parent should be able to pause processing of a child..example processing complete, delivery is next step, processing is paused by "awaiting payment" at the parent level. Whether this is done by something executing was the parent level that impacts the child level process or whether the child monitors for a status of awaiting payment might be different from an event such as a parent changing status to closed due to non-payment.

We need to have a detailed design session to address this before more development takes place as I believe we've been applying changes that might break when code is revised to enable this concept.

Also, we discussed the fact that there have been multiple iterations of gathering regulatory/statutory information specific to an driven configurability model starting with fee calculations. I might be wrong but I believe we then had another iteration for estimate rules. It seems we had a third iteration recently but at this point I don't recall exactly what all has happened. I pointed out yesterday that the concept of a sample has resulted in complications and effort that might have been minimized had we completed a "big bang" approach to research, attempting to gather all rules from all 50 states, and condensed down to a parameterized model. This could be divided into rules for fees/estimates, redaction, communications, calendar/tolling, and whatever else makes sense.

The result of the exercise would be a parameterized list for each category that I just mentioned. This would allow a change to the UI that is simpler to understand. A navigation label on the main panel would have a description such as Statutory/Regulatory Rules Configuration (i'm sure we can come up with a better description. Clicking takes user to a UI that has a button to access the associated library. Content is displayed in a window, a button to upload allows upload of documents. Some method of managing/deleting should be included.

Each category of rules lists described earlier (fees estimates, redaction, communications, etc) would be represented on the screen presented when clicking from the main nav panel. The UI would be designed with virtually the same content as the corresponding rules list, and this is the UI that would be used for configuration for that category. Click a button to begin presents existing configuration and revised configuration. Existing is for reference, display only. A button can be clicked to populate AI suggestions. Each individual item populated by AI could be accepted or manually corrected/modified. The only option for the user at that point is save as proposed revision. Someone with a lesser level of rights could do this, but the next step would be review/edit/submit by the individual person responsible for that category. That would be something entered during set up, requiring no judgement or determination by the system. The user name/email would be the "rights" rule. When that person submits, an attestation window pops up explaining that the city is responsible for accuracy and legal compliance of the configuration, with a button to go back to review or a button to acknowledge and submit.

I am currently working through a refined prompt and legal information gathering passes for a few states using chat gpt. I will upload the prompts and results later. My plan is to task multiple agents to work through state level information gathering, but the current iterations are helping me understand if limiting the scope resources to a defined type provides virtually the same results with less ai processing effort. It's also expanding the categories with each pass so that AI will be able to more cleanly assign with it determines to be a rule with generating output. My question for you....if I provide the script after I make a few more passes, can you review and launch one agent per state, and review output to identify any where it's likely that there was error and if so launch another? I would like the output for each state copied to git. Anyway, my question really is can you review/revise/test the prompt and if everything checks out, spawn an agent per state with the prompt without my involvement? And if so, can you copy the resulting data to my github? I will likely create another repo for this type of data.